

Domain-Driven Design & Event Storming

Ubiquitous Language · Value Objects · Aggregates · Repositories · Bounded Contexts · Event Storming

Pourquoi le DDD ?

Le constat d'Eric Evans (2003)

- Le logiciel doit incarner une riche connaissance du domaine
- L'approche CRUD modélise des opérations SQL ; l'approche orientée domaine modélise des comportements métier
- `Booking.confirm()` vs `UPDATE reservations SET status = 'confirmed'`

Ce que le matin a posé

- Le code doit parler un langage humain compréhensible
- L'après-midi pose la question : lequel ? Celui de la machine ou celui du domaine métier ?
- Le DDD est la réponse systématique à cette question

Deux niveaux complémentaires

- DDD stratégique : Ubiquitous Language, Bounded Contexts, Context Map — s'applique à tout projet non trivial
- DDD tactique : Entities, Value Objects, Aggregates, Repositories, Domain Services — pour les sous-domaines à forte complexité métier (Core Domain)

Session 3

DDD — Fondamentaux

Ubiquitous Language · Blocs tactiques · Bounded Contexts · Liens SOLID

Ubiquitous Language — Le langage partagé

Définition

Vocabulaire commun partagé par les développeurs et les experts métier d'un même contexte — utilisé à l'oral, dans les spécifications et directement dans le code (noms de classes, méthodes, variables).

Pourquoi c'est crucial

- Chaque traduction mentale entre langage métier et langage code est une source de friction et d'erreurs
- Un code qui parle Invoice, CreditNote, DueDate est compréhensible par un expert comptable
- Un code qui parle Doc, Entry, Date1 nécessite un dictionnaire oral non documenté
- Lien direct avec le nommage expressif vu ce matin

Comment le construire

- Ateliers de modélisation conjoints avec les experts métier (Event Storming)
- Glossaire vivant maintenu à jour, visible de tous
- Revue du langage dans le code lors des code reviews
- Refus des synonymes : Order et Purchase ne désignent pas la même chose dans le même contexte
- Refus des abréviations : mdp, cmd, usr hors du domaine

Value Objects & Entities

Value Object — Objet Valeur

- Définition : concept du domaine défini entièrement par ses attributs — pas d'identité propre
- Deux Value Objects avec les mêmes attributs sont interchangeables
- Immuable : ses attributs ne changent jamais après la création
- Auto-validant : les invariants sont vérifiés dans le constructeur
- Exemples : Money(100, 'EUR'), Email('alice@ex.com'), DateRange(start, end), PostalCode
- Elimine la Primitive Obsession : float ≠ Money, string ≠ Email

Entity — Entité

- Définition : objet dont l'identité est ce qui importe, indépendamment de ses attributs
- Possède un identifiant stable qui traverse son cycle de vie
- Mutable : ses attributs peuvent changer, mais son identité reste la même
- L'égalité est définie par l'identifiant, pas par les attributs
- Deux entités avec les mêmes attributs sont quand même distinctes si leurs ids diffèrent
- Exemple : Customer — même personne même si elle change de nom ou d'adresse

Aggregate — Le pattern le plus important du DDD tactique

Définition

Groupe d'entités et de Value Objects traité comme une unité de cohérence. L'une d'entre elles est désignée Aggregate Root. Toute modification passe par la racine.

Règle 1 — Accès par la racine uniquement

- La racine est la seule référence publiée à l'extérieur de l'agrégat
- Les entités internes ne sont pas accessibles directement depuis l'extérieur

Règle 2 — Une transaction, un agrégat

- Un seul agrégat est modifié par transaction
- La cohérence entre agrégats est éventuelle (eventual consistency)

Règle 3 — Références par identifiant

- Les références entre agrégats se font par identifiant (OrderId, CustomerId)
- Jamais par objet — évite le couplage en mémoire

Règle 4 — Taille minimale

- Un agrégat de 20 entités est un signal que les frontières sont mal tracées
- Vaughn Vernon : si deux concepts peuvent changer indépendamment, ils appartiennent à deux agrégats distincts

Repository & Domain Service

Repository — Dépôt

- Abstrait la persistance d'un agrégat — donne l'illusion que les agrégats vivent en mémoire
- Un Repository par Aggregate Root (pas par entité interne)
- L'interface est définie dans la couche domaine
- L'implémentation est dans la couche infrastructure (SQL, NoSQL, mémoire...)
- Les méthodes parlent le langage du domaine : `findByCustomer()`, pas `selectWhereCustomerId()`
- Lien direct avec le DIP : le domaine dépend de l'abstraction, l'infrastructure fournit la concrétion

Domain Service — Service de domaine

- Encapsule une opération métier significative qui ne trouve naturellement sa place dans aucune entité
- Généralement parce qu'elle implique plusieurs agrégats ou est sans état
- Exemple : `FundTransferService` — transfère des fonds entre deux comptes (deux agrégats)
- A ne pas confondre avec :
- → Application Service : orchestration des cas d'usage
- → Infrastructure Service : email, SMS, BDD (couche infra)

Bounded Contexts & DDD Stratégique

Frontières du modèle · Context Map · Anti-Corruption Layer · Tactique vs Stratégique

Bounded Context — Frontière du modèle

Définition

Frontière dans laquelle un modèle de domaine particulier s'applique de façon cohérente. A l'intérieur, chaque terme a une définition précise et unique. Un même mot peut avoir une signification différente dans un autre contexte.

Exemple — le mot « client » dans un système e-commerce

Ventes

- Personne qui passe une commande
- Attributs : nom, adresse livraison, historique achats

Service client

- Interlocuteur avec tickets de support
- Attributs : n° contrat, niveau SLA, historique incidents

Comptabilité

- Débiteur dans le grand livre
- Attributs : SIRET, RIB, conditions de paiement

Marketing

- Segment d'audience
- Attributs : comportement d'achat, score NPS, canal d'acquisition

Context Map & Anti-Corruption Layer

Context Map — Types de relations

- Shared Kernel : modèle commun partagé — couplage fort, accord des deux équipes requis
- Customer-Supplier : l'aval dépend de l'amont qui définit le contrat
- Conformist : l'aval adopte le modèle de l'amont sans négociation (API bancaire, ERP imposé)
- Open Host Service / Published Language : API publique documentée (contrat OpenAPI)

Anti-Corruption Layer (ACL)

- Couche de traduction qui protège l'intégrité du modèle face aux systèmes externes
- L'aval traduit le modèle de l'amont en son propre langage — le domaine ne connaît jamais le système tiers directement
- Pattern le plus protecteur de l'intégrité du modèle
- Lien SOLID : application directe du DIP + LSP

Pourquoi cartographier les contextes ?

- Rend visible les dépendances entre équipes et systèmes — souvent implicites et sources de conflits
- Identifie les risques de couplage accidentel entre des domaines qui devraient être indépendants
- Permet de décider : partager le code, définir un contrat, ou isoler derrière un ACL

DDD Tactique vs Stratégique — Quand l'appliquer

Type de sous-domaine	Description	Approche recommandée
Core Domain	Avantage concurrentiel différenciant	DDD tactique complet
Supporting Domain	Soutient le core, pas différenciant	DDD tactique partiel ou CRUD enrichi
Generic Domain	Commodité standard (auth, facturation simple)	Solution standard du marché ou CRUD simple

Liens DDD ↔ SOLID

- Aggregate ↔ SRP : l'agrégat définit une frontière de cohérence — une seule raison de changer
- Repository ↔ DIP : l'interface est dans le domaine, l'implémentation dans l'infrastructure
- Value Object ↔ OCP + SRP : immuable et auto-validant — fermé à la modification après création
- Bounded Context ↔ ISP : chaque contexte expose uniquement le modèle pertinent pour ses consommateurs
- Anti-Corruption Layer ↔ DIP + LSP : le domaine dépend d'une abstraction, pas du système tiers

Session 4

Event Storming & Modélisation du domaine

Atelier collaboratif · Post-its · Du mur au code squelette

Event Storming — Origine & intention (Alberto Brandolini, 2013)

Définition

Technique d'atelier collaboratif : explorer rapidement un domaine métier complexe en réunissant développeurs et experts métier autour de post-its et d'un grand mur.

Le principe fondateur de Brandolini

- « La quantité de connaissances du domaine contenue dans la tête des experts métier est toujours supérieure à ce que peut absorber n'importe quel développeur seul »
- L'Event Storming permet de les externaliser collectivement
- Délibérément non structuré dans ses premières phases : on libère la parole avant de structurer

Ce que l'Event Storming produit

- Un modèle collaboratif du domaine compris par tous
- L'Ubiquitous Language émergé organiquement
- Les frontières naturelles des Bounded Contexts
- Les règles métier (Policies) rendues visibles
- Un squelette de code directement déductible des post-its

Les 8 types de post-its — Convention de Brandolini

Domain Event (orange)

Quelque chose qui s'est passé, formulé au passé

Ex. : Commande confirmée

Command (bleu)

Ce qui déclenche un événement, à l'impératif

Ex. : Confirmer la commande

Actor (jaune clair)

Humain ou système qui émet la commande

Ex. : Client, Système paiement

Aggregate (jaune)

Entité métier sur laquelle s'exerce la commande

Ex. : Order, Reservation

Policy / Reaction (violet)

Règle déclenchée par un événement

Ex. : Quand commande créée → vérifier stock

Hot Spot (rouge)

Question ou ambiguïté à résoudre

Ex. : Que se passe-t-il si le stock est à 0 ?

Read Model (vert)

Information consultée pour prendre une décision

Ex. : Liste des produits disponibles

External System (gris)

Système externe impliqué dans le flux

Ex. : Passerelle de paiement, ERP

Les Domain Events (orange) sont posés en premier, sans contrainte d'ordre, par tous les participants simultanément.

Déroulement d'un atelier Event Storming — 4 phases

Phase 1 — Divergence (20 min)

- Chaque participant écrit des Domain Events sur post-its orange
- Sans contrainte d'ordre ni de complétude
- Formuler au passé composé — minimum 10 événements par groupe
- On pose tous les post-its sans discussion

Phase 2 — Convergence (20 min)

- Trier les post-its pour former une ligne de temps
- Regrouper les doublons
- Les contradictions donnent lieu à un Hot Spot (rouge)
- Identifier les événements manquants

Phase 3 — Enrichissement (30 min)

- Pour chaque événement significatif :
- → Qui l'a déclenché ?
- → Command + Actor
- → Sur quelle entité ?
- → Aggregate
- → Quelle réaction ? → Policy

Phase 4 — Frontières (20 min)

- Tracer des lignes autour des groupes partageant le même langage et les mêmes règles
- Chaque groupe = candidat Bounded Context
- Nommer chaque contexte avec l'Ubiquitous Language

Du mur de post-its au code squelette — Mapping direct

Post-it Event Storming	Concept DDD	Implémentation
Domain Event (orange)	DomainEvent	Classe immuable avec timestamp
Aggregate (jaune)	Aggregate Root	Classe avec méthodes métier et invariants
Command (bleu)	Méthode de l'agrégat / Application Service	Méthode ou classe de commande
Policy (violet)	DomainEventHandler	Abonné au bus d'événements
Read Model (vert)	Query / DTO	Projection de lecture
Bounded Context	Module / Namespace	Organisation de répertoires et de packages

CRUD classique vs Approche orientée domaine

Approche CRUD

- La logique métier vit dans le contrôleur — impossible à tester sans HTTP et BDD
- Pas d'Ubiquitous Language : `status = 'confirmed'` plutôt que `order.confirm()`
- Les effets de bord (email, stock) sont mélangés à la transaction
- Modifier une règle de confirmation = modifier le contrôleur
- Adapté aux : Generic Domains, formulaires simples, interfaces CRUD pures

Approche orientée domaine

- `order.confirm()` est testable en isolation — pas de BDD, pas de HTTP
- La règle métier vit dans l'agrégat — le contrôleur/handler est mince
- L'email est découplé de la transaction via un Domain Event
- Modifier la règle de confirmation = modifier uniquement Order
- Adapté aux : Core Domains à forte complexité métier

Résumé

Concept	Ce qu'il apporte	Relation avec le matin
Ubiquitous Language	Pont entre code et métier	Prolonge le nommage expressif (Clean Code)
Value Object	Encapsulation des règles de validation	Elimine la Primitive Obsession
Entity	Identité stable, mutations contrôlées	Applique SRP et POLA
Aggregate	Frontière de cohérence transactionnelle	Applique SRP, élimine les incohérences
Repository	Abstraction de la persistance	Application directe du DIP
Domain Service	Opérations impliquant plusieurs agrégats	Application du SRP
Bounded Context	Frontière du modèle et du langage	Application de l'ISP à l'échelle du système
Anti-Corruption Layer	Protection face aux systèmes externes	Application du DIP + LSP
Event Storming	Exploration collaborative du domaine	Produit l'Ubiquitous Language organiquement
Domain Events	Communication découplée entre contextes	Prépare CQRS et Event Sourcing (jour 2)